

Report Pratica

Simulazione di un attacco Brute-Force a un servizio SSH

Consegna

Si richiede allo studente di scrivere un programma, con un linguaggio a sua scelta tra Python e C, che permetta l'esecuzione di un attacco Brute-Force ad un servizio SSH su una macchina Debian/Ubuntu (Kali va benissimo come macchina di test).

Approccio adottato

Ho scelto di sviluppare il programma in Python (script `ssh_bruteforce1.py`), realizzando una versione didattica e sicura dell'attacco: anziché connettersi realmente a un servizio SSH in ascolto, lo script simula il processo di autenticazione tramite una funzione `verifica_login` che confronta le credenziali fornite con una coppia username/password corretta definita internamente, riproducendo la logica di un tentativo di accesso senza generare traffico di rete reale verso un demone SSH.

Struttura dello script

Lo script definisce le credenziali corrette del laboratorio (username "studente", password "segreta123") e una wordlist di 8 password comuni da provare in sequenza. La funzione `verifica_login` confronta username e password ricevuti con quelli corretti, restituendo True o False:

```
ssh_bruteforce1.py > brute_force
1 import time
2
3 # =====
4 # SSH BRUTE-FORCE - LABORATORIO DIDATTICO
5 # =====
6
7 # Credenziali corrette del laboratorio
8 USERNAME_CORRETTO = "studente"
9 PASSWORD_CORRETTA = "segreta123"
10
11 # Wordlist del laboratorio
12 PASSWORDS = [
13     "123456",
14     "password",
15     "admin",
16     "admin123",
17     "qwerty",
18     "test123",
19     "segreta",
20     "segreta123"
21 ]
22
23
24 def verifica_login(username, password):
25     """
26     Simula l'autenticazione SSH.
27     """
28
29     if username == USERNAME_CORRETTO and password == PASSWORD_CORRETTA:
30         return True
31     return False
32
33
34 def brute_force(username):
35     """
36     Prova tutte le password presenti nella wordlist.
37     """
38
39
40     tentativi = 0
41     inizio = time.time()
42
```

La funzione `brute_force` riceve lo username target e itera su tutte le password della wordlist, contando i tentativi effettuati e misurando il tempo trascorso con il modulo `time`. Per ogni password provata viene stampato il progresso ("Tentativo N/8: password"); se la verifica ha successo, vengono mostrati un riepilogo con username, password trovata, numero di tentativi e tempo impiegato:

```
12 print("\n=====")
13 print("      SSH BRUTE-FORCE - LABORATORIO")
14 print("=====")
15 print(f"Username target: {username}")
16 print(f"Password da provare: {len(PASSWORDS)}")
17 print("-----")
18
19 for password in PASSWORDS:
20     tentativi += 1
21
22     print(
23         f"Tentativo {tentativi}/{len(PASSWORDS)}: "
24         f"{password}"
25     )
26
27     # Verifica della password
28     if verifica_login(username, password):
29
30         fine = time.time()
31         tempo = fine - inizio
32
33         print("\n=====")
34         print("      PASSWORD TROVATA")
35         print("=====")
36         print(f"Username : {username}")
37         print(f>Password : {password}")
38         print(f"Tentativi: {tentativi}")
39         print(f"Tempo    : {tempo:.4f} secondi")
40         print("=====")
41
42         return
43
44 # Se nessuna password è corretta
45 fine = time.time()
46 tempo = fine - inizio
47
48 print("\n=====")
49 print("      PASSWORD NON TROVATA")
```

Se invece nessuna password della wordlist risulta corretta, lo script stampa un messaggio di insuccesso con il numero di tentativi effettuati e il tempo totale impiegato. La funzione `main` gestisce l'input dello username da riga di comando, validandolo prima di avviare l'attacco:

```

78     tempo = time - inizio
79
80     print("\n=====")
81     print("          PASSWORD NON TROVATA")
82     print("=====")
83     print(f"Tentativi: {tentativi}")
84     print(f"Tempo      : {tempo:.4f} secondi")
85     print("=====")
86
87
88 def main():
89
90     print("=====")
91     print("          SIMULATORE ATTACCO SSH")
92     print("=====")
93
94     username = input("Inserisci username: ").strip()
95
96     if username == "":
97         print("Errore: username non valido.")
98         return
99
100     brute_force(username)
101
102
103 if __name__ == "__main__":
104     main()

```

Esecuzione dello script

Ho eseguito lo script da terminale (python ssh_bruteforce1.py), inserendo come username target "Godfred". Non essendo presente nella wordlist la password corretta ("segreta123" è associata solo allo username "studente"), l'attacco ha provato tutte e 8 le password disponibili senza trovare una corrispondenza, terminando con l'esito "PASSWORD NON TROVATA" in un tempo trascurabile (0.0001 secondi), coerente con una simulazione locale priva di latenza di rete:

```
Output Debug Console Terminal Ports
(kali@kali) - [~/Desktop/Epicode/Brute_Force]
$ python ssh_bruteforce1.py
=====
SIMULATORE ATTACCO SSH
=====
Inserisci username: Godfred

=====
SSH BRUTE-FORCE - LABORATORIO
=====
Username target: Godfred
Password da provare: 8
-----
Tentativo 1/8: 123456
Tentativo 2/8: password
Tentativo 3/8: admin
Tentativo 4/8: admin123
Tentativo 5/8: qwerty
Tentativo 6/8: test123
Tentativo 7/8: segreta
Tentativo 8/8: segreta123

=====
PASSWORD NON TROVATA
=====
Tentativi: 8
Tempo : 0.0001 secondi
=====
```

Considerazioni etiche e limiti dello script

Va sottolineato che questa implementazione ha uno scopo puramente didattico: la funzione `verifica_login` non si connette realmente ad alcun servizio SSH, ma simula il confronto delle credenziali in locale. Un vero attacco brute-force a un servizio SSH reale (ad esempio tramite la libreria `paramiko` in Python) dovrebbe essere eseguito esclusivamente in un ambiente di laboratorio controllato e isolato, su macchine di cui si ha esplicita autorizzazione, in quanto tentativi non autorizzati contro sistemi altrui costituiscono un reato.

Conclusioni

Attraverso questo esercizio ho imparato a:

- Strutturare in Python la logica di un attacco a dizionario (dictionary attack), iterando su una wordlist di credenziali
- Misurare tempo di esecuzione e numero di tentativi con il modulo `time`
- Simulare in modo sicuro il comportamento di un servizio di autenticazione per scopi di studio, senza generare traffico verso servizi reali
- Riflettere sui limiti legali ed etici di questo tipo di tecniche, da utilizzare solo in ambienti di test autorizzati